application is entered. They are the core components of applications built with Django. Views take a web request and return a response to the user. Views are meant to perform various operations such as fetching objects from the database, modifying the objects, rendering forms, returning HTML files and so on.

Views request information from the model in the MVT architecture and pass it on to a template. The following chapter will be completely focused upon templates.

'Views.py' files are used to store views.

A view is the web page of a Django application that serves a specific function and has a specific template.

For example, in a blog application, there can be the following views:

- Blog homepage: It displays the latest entries.
- Entry "detail" page: It is a permalink page for a single entry.
- Year-based archive page: It displays all the months with entries for a given year.
- Month-based archive page: It displays all the days with entries for a given month.
- Day-based archive page: It displays all entries for a given day.
- Comment action: It enables posting comments to a given entry.

Another example is a poll application, where there are the following four views:

- Question "index" page: It displays the latest questions.
- Question "detail" page: It displays a question text, without results but with a form to vote.
- Question "results" page: It displays results for a particular question.
- Vote action: It handles voting for a particular option in a particular question.

In Django, web pages and other web content are delivered with the help of views. Each view is represented with a Python function (or method, in the class-based views). Django chooses a view by examining the requested URL (or the part of the URL after the domain name).

Types of Views

Based on the core functionalities, Views are broadly divided into two categories:

i. Class-Based Views (CBVs)
ii. Function-Based Views (FBVs)

Let's compare them.